# Cluster Architecture

This page describes how the nodes or clusters in Weaviate's replication design behave.

In Weaviate, metadata replication and data replication are separate. For the metadata, Weaviate uses the Raft consensus algorithm. For data replication, Weaviate uses a leaderless design with eventual consistency.

## Node Discovery

By default, Weaviate nodes in a cluster use a gossip-like protocol through Hashicorp's Memberlist to communicate node state and failure scenarios.

Weaviate is optimized to run on Kubernetes, especially when operating as a cluster. The Weaviate Helm chart makes use of a `StatefulSet` and a headless `Service` that automatically configures node discovery.

▸ FQDN for node discovery

## Metadata replication: Raft

Weaviate uses the Raft consensus algorithm for metadata replication, implemented with Hashicorp's raft library. Metadata in this context includes collection definition and shard/tenant states.

Raft ensures that metadata changes are consistent across the cluster. A metadata change is forwarded to the leader node, which applies the change to its log before replicating it to the follower nodes. Once a majority of nodes have acknowledged the change, the leader commits the change to the log. The leader then notifies the followers, which apply the change to their logs.

This architecture ensures that metadata changes are consistent across the cluster, even in the event of (a minority of) node failures.

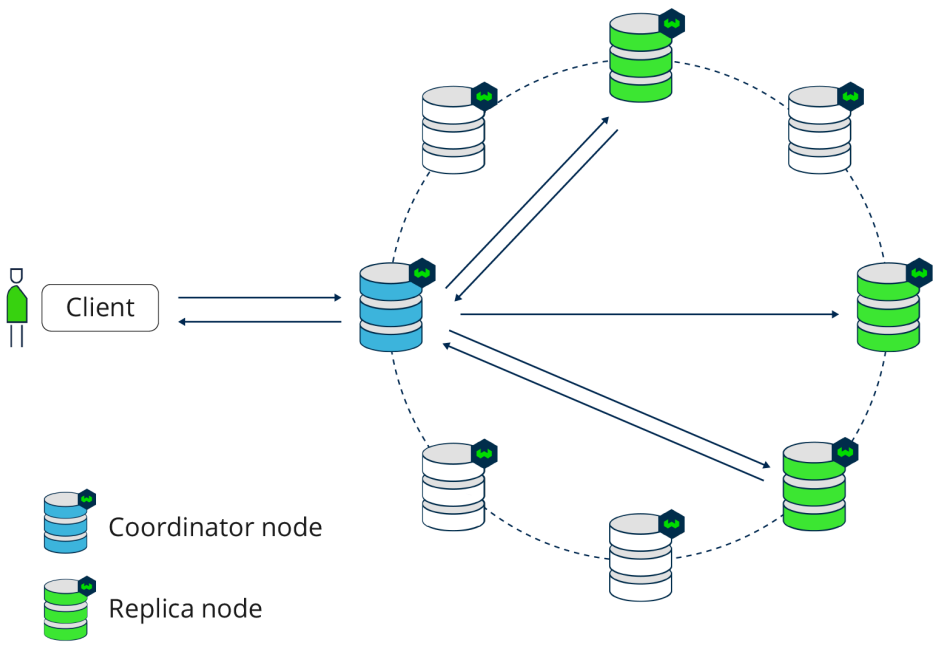
As a result, a Weaviate cluster will include a leader node that is responsible for metadata changes. The leader node is elected by the Raft algorithm and is responsible for coordinating metadata changes.

## Data replication: Leaderless

Weaviate uses a leaderless architecture for data replication. This means there is no central leader or primary node that will replicate to follower nodes. Instead, all nodes can accept writes and reads from the client, which can offer better availability. There is no single point of failure. A leaderless replication approach, also known as Dynamo-style data replication (after Amazon's implementation), has been adopted by other open-source projects like Apache Cassandra.

In Weaviate, a coordination pattern is used to relay a client's read and write requests to the correct nodes. Unlike in a leader-based database, a coordinator node does not enforce any ordering of the operations.

The following illustration shows a leaderless replication design in Weaviate. There is one coordination node, which leads traffic from the client to the correct replicas. There is nothing special about this node; it was chosen to be the coordinator because this node received the request from the load balancer. A future request for the same data may be coordinated by a different node.




The main advantage of a leaderless replication design is improved fault tolerance. Without a leader that handles all requests, a leaderless design offers better availability. In a single-leader design, all writes need to be processed by this leader. If this node cannot be

reached or goes down, no writes can be processed. With a leaderless design, all nodes can receive write operations, so there is no risk of one master node failing.

On the flipside of high availability, a leaderless database tends to be less consistent. Because there is no leader node, data on different nodes may temporarily be out of date. Leaderless databases tend to be eventually consistent. Consistency in Weaviate is tunable, but this occurs at the expense of availability.

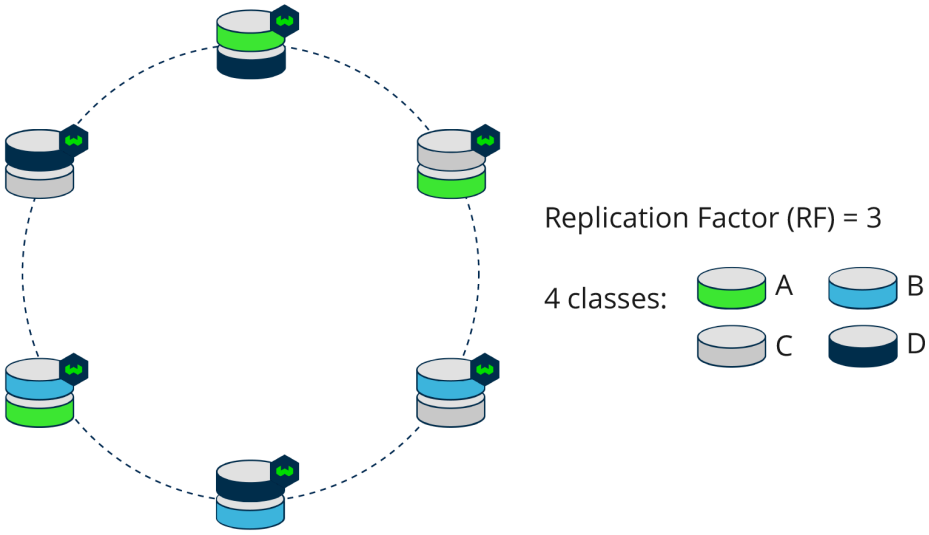
## Replication Factor

> **REPLICATION FACTOR CHANGE**
>
> The replication factor of a collection cannot be updated by updating the collection's definition.
>
> From `v1.32` by using replica movement, the replication factor of a shard can be changed.

In Weaviate, data replication is enabled and controlled per collection. This means you can have different replication factors for different collections.

The replication factor (RF or n) determines how many copies of data are stored in the distributed setup. A replication factor of 1 means that there is only 1 copy of each data entry in the database setup, in other words there is no replication. A replication factor of 2 means that there are two copies of each data entry, which are present on two different nodes (replicas). Naturally, the replication factor cannot be higher than the number of nodes. Any node in the cluster can act as a coordinating node to lead queries to the correct target node(s).

A replication factor of 3 is commonly used, since this provides a right balance between performance and fault tolerance. An odd number of nodes is generally preferred, as it makes it easier to resolve conflicts. In a 3-node setup, a quorum can be reached with 2 nodes. Therefore the fault tolerance is 1 node. In a 2-node setup, on the other hand, no node failures can be tolerated while still reaching consensus across nodes. In a 4-node setup, respectively, 3 nodes would be required to reach a consensus. Thus, a 3-node setup has a better fault-tolerance to cost ratio than either a 2-node or 4-node setup.



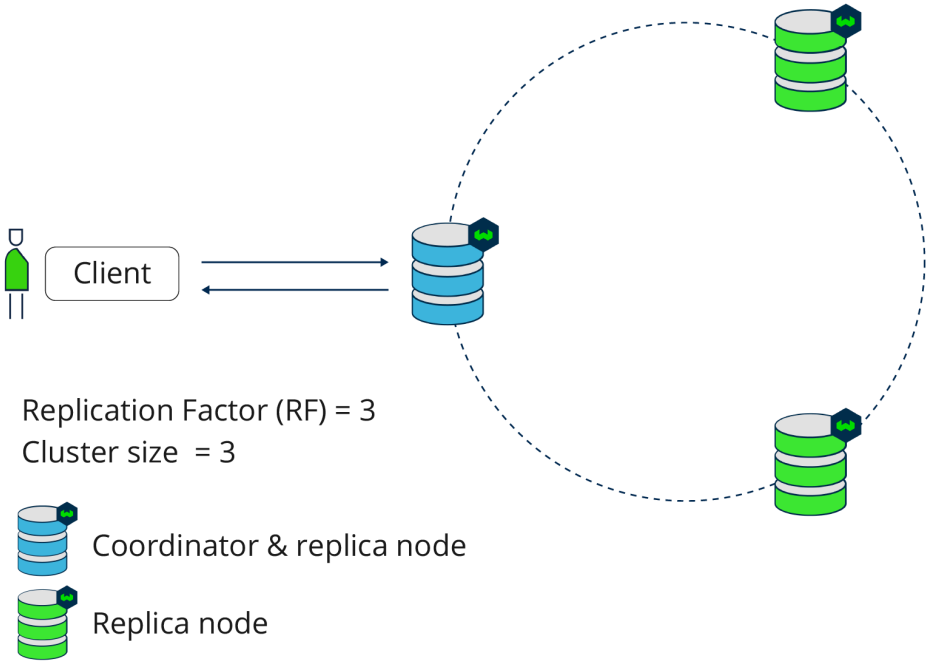

## Write operations

On a write operation, the client's request will be sent to any node in the cluster. The first node which receives the request is assigned as the coordinator. The coordinator node sends the request to a number of predefined replicas and returns the result to the client. So, any node in the cluster can be a coordinator node. A client will only have direct contact with this coordinator node. Before sending the result back to the client, the coordinator node waits for a number of write acknowledgments from different nodes depending on the configuration. How many acknowledgments Weaviate waits for, depends on the consistency configuration.

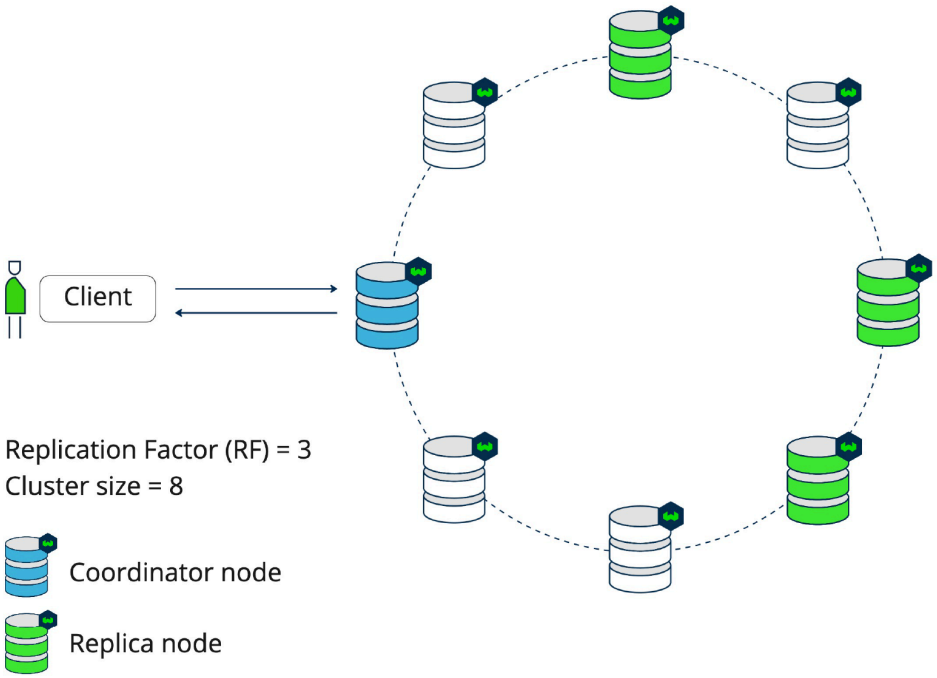
### Steps

1. The client sends data to any node, which will be assigned as the coordinator node
2. The coordinator node sends the data to more than one replica node in the cluster
3. The coordinator node waits for acknowledgment from a specified proportion (let's call it `x`) of cluster nodes. Starting with v1.18, `x` is configurable, and defaults to `QUORUM` nodes.
4. When `x` ACKs are received by the coordinator node, the write is successful.

As an example, consider a cluster size of 3 with replication factor of 3. So, all nodes in the distributed setup contain a copy of the data. When the client sends new data, this will be replicated to all three nodes.




With a cluster size of 8 and a replication factor of 3, a write operation will not be sent to all 8 nodes, but only to those three containing the replicas. The coordinating node will determine which nodes the data will be written to. Which nodes store which collections (and therefore shards) is determined by the setup of Weaviate, which is known by each node and thus each coordinator node. Where something is replicated is deterministic, so all nodes know on which shard which data will land.




## Read operations

Read operations are also coordinated by a coordinator node, which directs a query to the correct nodes that contain the data. Since one or more nodes may contain old (stale) data, the read client will determine which of the received data is the most recent before sending it to the user.

### Steps

1. The client sends a query to Weaviate, any node in the cluster that receives the request first will act as the coordinator node
2. The coordinator node sends the query to more than one replica node in the cluster
3. The coordinator waits for a response from x nodes. *x is configurable (`ALL`, `QUORUM` or `ONE`, available from v1.18, Get-Object-By-ID type requests have tunable consistency from v1.17).*
4. The coordinator node resolves conflicting data using some metadata (e.g. timestamp, id, version number)
5. The coordinator returns the latest data to the client

If the cluster size is 3 and the replication factor is also 3, then all nodes can serve the query. The consistency level determines how many nodes will be queried.

If the cluster size is 10 and the replication factor is 3, the 3 nodes which contain that data (collection) can serve queries, coordinated by the coordinator node. The client waits until x (the consistency level) nodes have responded.

## Questions and feedback

If you have any questions or feedback, let us know in the user forum.

### Technical questions

If you have questions feel free to post on our Community forum.

### Documentation feedback

Leave feedback by opening a GitHub issue.



**Documentation**

Weaviate Database

Deployment documentation

Weaviate Cloud

Weaviate Agents

**Support**

Forum

Was this page helpful?

Yes No